

CIA-3 Project Guide — Mission Crisis: Disaster Response

"Classify and Prioritise Incoming Emergency-Help Requests During Natural Disasters"

This guide maps every step to your rubric (Q1–Q5, 25 marks total) so you can work through it section by section and know exactly what's expected.

1. Recommended Dataset

Primary recommendation: Disaster Response Messages (Figure Eight / Appen, via Robert Munro)

- **What it is:** ~25,000–30,000 real messages sent during actual disasters — 2010 Haiti earthquake, 2010 Pakistan floods, 2012 Superstorm Sandy, plus news articles covering hundreds of other disasters.
- **Labels:** Each message is tagged across **36–38 categories** such as `request`, `aid_related`, `medical_help`, `search_and_rescue`, `water`, `food`, `shelter`, `death`, `infrastructure_related`, `floods`, `earthquake`, etc. Also includes a `genre` field (`direct`, `news`, `social`) and (in the original release) an `actionable` label for the Haiti subset — exactly the "prioritise" angle your brief asks for.
- **PII:** Already stripped of personally identifying information by the original curators — satisfies your "no PII" instruction out of the box.
- **Where to get it:**
 - GitHub (cleanest, with train/val/test splits already made):
`github.com/rmunro/disaster_response_messages`
 - Kaggle mirror: "Multilingual Disaster Response Messages"
`kaggle.com/datasets/landlord/multilingual-disaster-response-messages`
- **Cite it as:** Munro, R. & Xiao, J. (Figure Eight/Appen). *Disaster Response Messages Dataset*. Originally released via Figure Eight; derived in part from Munro & Manning, CoNLL 2011 ("Short Message Communications...").

Why this dataset fits your brief so well

- It is **literally** "incoming emergency-help requests during a disaster" — not a proxy.
- It naturally supports a **prioritisation reframe**: you don't have to use all 36 raw labels as-is (that's a multi-label NLP problem, harder to shoehorn into the Decision Tree → RF → Boosting → Stacking rubric). Instead, **engineer a single priority target**, e.g.:
 - `is_actionable` / `high_priority` = 1 if message touches `medical_help`, `search_and_rescue`, `death`, `security`, `fire`, `water`, `food`, `shelter` (life-critical needs) else 0.
 - Or a 3-class **priority tier**: `Critical` (medical/rescue/death), `High` (water/food/shelter/infrastructure), `Low` (general/news/other).

- This turns it into a clean **classification problem** your ensemble pipeline can directly target.

Two solid alternatives (in case you want something simpler or more tabular)

Dataset	Nature	Trade-off
HumAID (CrisisNLP, Alam et al.)	Tweets labelled with 10 humanitarian classes (e.g. <code>injured_or_dead_people</code> , <code>rescue_volunteering_or_donation_effort</code> , <code>requests_or_urgent_needs</code>) across 19 real disaster events	Cleaner category set for "prioritisation," but you'll need to derive text features yourself; less document support than Figure Eight
"Real or Not? NLP with Disaster Tweets" (Kaggle)	~10,000 tweets, binary: disaster vs. not	Very easy to start with, well-documented, huge community support — but it answers "is this a real disaster report?" not "how urgent is this?", which is a weaker fit for the <i>prioritise</i> requirement in your brief

My recommendation: go with Disaster Response Messages + engineered priority target.

It gives you a defensible, real-world "prediction target" for Q1, plenty of raw signal for feature engineering in Q2, and enough data volume for a meaningful train/val/test split in Q3.

2. Q1 — Real-World Impact Framing (5 marks)

Write this as a short problem brief (½–1 page). Structure it like this:

1. **Problem statement:** During a disaster, thousands of messages (SMS, social media, direct reports) flood in simultaneously. Responders can't manually triage all of them fast enough — some are life-critical (trapped people, medical emergencies), others are lower urgency (general news, requests for non-essential supplies). Misclassifying a critical message as low priority costs lives; misclassifying everything as critical causes alert fatigue and wastes scarce responder time.
2. **Beneficiaries:** Disaster-response NGOs/agencies (e.g. Red Cross-type coordination teams), field responders, and — indirectly — the affected population whose critical needs get triaged faster.

3. **Prediction target:** State your engineered target explicitly, e.g. *"predict whether an incoming message represents a Critical / High / Low priority need, based on the message text and derived features."*
 4. **Why ML is suitable:** Volume and speed — no team can manually read tens of thousands of messages in real time; patterns in language (keywords, urgency phrasing, co-occurring needs) are learnable; ensembles improve robustness over any single rule-based keyword filter.
 5. **Dataset & unit of analysis:** One dataset row = one message. Cite the source (see above).
 6. **Responsible-use limitations (important — examiners will look for this):**
 - Labels reflect *specific past disasters* (Haiti 2010, Pakistan 2010, Sandy 2012) — language patterns, dialects, and what counted as "actionable" were time- and context-specific; a model trained here may not transfer cleanly to a new disaster type or region.
 - Never should this fully **replace** human triage — treat it as a decision-support ranking tool, not an autonomous responder.
 - False negatives (a critical message marked low priority) are far costlier than false positives here — mention this now, you'll need it again in Q4.
-

3. Q2 — Data Wrangling & Feature Engineering (6 marks)

3.1 Load & audit

- Merge messages + categories files (if using the raw Figure Eight split) on `(id)`.
- Audit: missing values, duplicate messages, rows with all-zero/garbage category encodings (a known quirk in this dataset — some `(related)` values appear as `(2)` instead of 0/1; document and handle this explicitly, it's a great thing to mention as a "defensible preprocessing decision").
- Check `(genre)` distribution (`(direct)` vs `(news)` vs `(social)`) — news articles behave very differently from direct emergency messages; consider whether to keep, drop, or weight by genre.

3.2 Build the priority target

- Define the label mapping (Critical / High / Low) as described in Q1. Write a short justification for which categories map to which tier (this is itself a "domain-informed feature engineering" decision worth documenting).
- Check class balance after this transformation — it will likely be imbalanced (few Critical vs many Low/general messages). Note it now, handle it formally in 3.4.

3.3 Feature engineering (this is where "domain-informed" features earn marks)

Because raw input is text, you engineer a mix of **NLP-derived tabular features** so tree ensembles (RF/XGBoost/etc.) can use them directly — this avoids you having to justify deep learning, which isn't asked for:

- **TF-IDF vectors** (top-N terms, e.g. 300–500 dimensions) as the base text representation.
- **Engineered domain features:**
 - Message length (word/char count)
 - Presence of urgency keywords (`help`, `trapped`, `dying`, `urgent`, `now`, `SOS`) — count or binary flags
 - Presence of specific need keywords (`water`, `food`, `medicine`, `shelter`)
 - Punctuation intensity (`!` count, ALL-CAPS ratio) — often correlates with urgency in real messages
 - `genre` (one-hot: direct/news/social)
 - Sentiment/negativity score (optional, via a lightweight lexicon — VADER is fine and simple to justify)
- **Categorical encoding:** one-hot for `genre`; target/frequency encoding is overkill here — justify one-hot as more transparent/explainable (ties into Q4).
- **Numerical scaling:** StandardScaler or MinMax for engineered numeric features (TF-IDF is already normalized, so scale only the hand-built numeric features) — needed for Logistic Regression baseline especially.
- **Class imbalance:** handle with `class_weight='balanced'` in baseline and RF, and `scale_pos_weight` (XGBoost) or SMOTE on the *training fold only* (never on validation/test — explicitly say this out loud in your report, it's exactly the "leakage-safe" requirement the rubric names).

3.4 Leakage-safe pipeline

- Use `sklearn.pipeline.Pipeline` (or `ColumnTransformer` + `Pipeline`) so TF-IDF fitting, scaling, and imputation all happen **inside** cross-validation folds — never fit any transformer on the full dataset before splitting.
- Split: **stratified train/val/test**, e.g. 70/15/15, stratified on your priority label to preserve class balance across splits. Set a fixed `random_state` for reproducibility.

3.5 EDA to include

- Class distribution bar chart (Critical/High/Low)
 - Top TF-IDF terms per class
 - Message-length distribution by class
 - Genre × priority cross-tab
-

4. Q3 — Ensemble Architecture, Tuning & Comparison (8 marks — biggest chunk, spend the most time here)

Build these on the **exact same train/val/test split**, using the pipeline from Q2.

Model	Role	Notes
Decision Tree or Logistic Regression	Baseline	Keep it simple, shallow depth or default regularization — this is your reference point
Random Forest	Bagging	Tune <code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_leaf</code> , <code>class_weight</code>
XGBoost / LightGBM / CatBoost / AdaBoost	Boosting	Tune <code>learning_rate</code> , <code>n_estimators</code> , <code>max_depth</code> , <code>scale_pos_weight</code> (or equivalent)
Stacking or Voting Classifier	Heterogeneous ensemble	Combine e.g. Logistic Regression + Random Forest + XGBoost as base learners, with a simple meta-learner (Logistic Regression) on top

Tuning: Use `GridSearchCV` or `RandomizedSearchCV` with **stratified k-fold CV** (k=5) on the *training set only*. Never touch the test set during tuning.

Stacking-specific leakage warning (rubric explicitly calls this out): When building the stacking ensemble, generate the meta-features via **out-of-fold predictions**

(`StackingClassifier` in sklearn does this correctly by default — just don't hand-roll it by predicting on the same data the base models were trained on).

Evaluation — one held-out test set, same for every model:

- ROC-AUC (one-vs-rest, since you likely have 3 classes)
- Macro-F1 (important for imbalanced multi-class — don't just report accuracy)
- Precision & Recall **per class**, with special attention to **Recall on the "Critical" class** — this is your key business metric, tie it back to the false-negative cost argument from Q1
- Confusion matrix (all models, side-by-side)
- A summary table: baseline vs. bagging vs. boosting vs. stacking, and a clear written verdict on whether the best ensemble beats the baseline and by how much.

5. Q4 — Explainability & Ethics (4 marks)

- **Global explainability:** SHAP summary plot (beeswarm) on your best model — identify top features driving "Critical" predictions (e.g. urgency keywords, TF-IDF terms like "trapped"/"medical", message length).
 - **Local explainability:** Pick 2-3 individual predictions (including at least one the model got *wrong*) and show a SHAP force plot / LIME explanation — interpret in plain domain language ("this message was flagged Critical mainly because it contained 'trapped' and 'help now', and came through the direct-message genre").
 - **Ethics/fairness discussion to write:**
 - **Bias:** dataset messages are from specific 2010–2012 events, translated to English — language/dialect bias, possible under-representation of certain regions or communication styles.
 - **Privacy:** even though PII is stripped, be explicit that any real deployment must have a strict PII pipeline before ingestion.
 - **Uncertainty:** report predicted probabilities, not just hard labels — flag low-confidence predictions for mandatory human review.
 - **Cost asymmetry:** false negative (missed Critical case) vs. false positive (wasted responder attention) — argue why you'd tune the decision threshold to favor recall on Critical.
 - **Human oversight & deployment limits:** this should be a decision-support ranking tool for human triagers, not an autonomous dispatch system; recommend periodic retraining as language/events evolve.
-

6. Q5 — Three-Minute Pitch & Live Demo (2 marks)

Follow the timing exactly as specified:

- **0:00–0:35:** State the problem (message overload during disasters) and beneficiaries (response orgs, affected people).
- **0:35–1:15:** Show a quick EDA visual + describe your cleaning/feature engineering choices (priority-label construction, TF-IDF + urgency keyword features).
- **1:15–2:10:** Show your results table — baseline vs RF vs boosting vs stacking, highlight the winning model and its Recall on "Critical."
- **2:10–3:00:** Live demo — type or paste a **synthetic** message (e.g. *"Family of 4 trapped under debris near river bridge, need rescue urgently"*), show the predicted priority + SHAP/LIME local explanation, then close with one ethics/limitation line.

Keep code and a results chart visible on screen throughout — the rubric explicitly asks for this.

7. Suggested Toolkit

```
pandas, numpy          - data wrangling
scikit-learn            - pipelines, Decision Tree, Logistic Regression,
Random Forest,
                        AdaBoost, StackingClassifier/VotingClassifier,
metrics, CV
xgboost / lightgbm / catboost - boosting model (pick one)
imbalanced-learn        - SMOTE (inside pipeline, train-fold only)
shap, lime              - explainability
matplotlib, seaborn     - EDA & result visuals
nltk or vaderSentiment  - optional sentiment/urgency lexicon features
```

8. Suggested Timeline (if you have ~2 weeks)

Days	Task
1-2	Download dataset, write Q1 impact framing, initial EDA
3-5	Build leakage-safe pipeline, engineer features, finalize priority label (Q2)
6-9	Baseline → RF → boosting → stacking, tuning, comparison table (Q3)
10-11	SHAP/LIME explainability + ethics write-up (Q4)
12	Record 3-minute video (Q5)
13-14	README, reproducibility check, final polish, buffer

9. Reproducibility Checklist for Submission

- ☐ Codebase/notebook (clean, with markdown commentary matching Q1-Q4 sections)
- ☐ README with exact run steps + dataset citation/link
- ☐ Saved trained pipeline (`.pkl`/`.joblib`) or a script that reproduces training end-to-end
- ☐ Results/figures folder (confusion matrices, ROC curves, SHAP plots)
- ☐ Ethics statement (can reuse your Q4 write-up)
- ☐ 3-minute pitch-and-demo video